

# Software Requirements Specification

for *RailReserve*

Version 4

Prepared by Jan Kośliński

14.04.2026

## Table of Contents

|                                                 |   |
|-------------------------------------------------|---|
| 1. Introduction .....                           | 3 |
| 1.1. Product Scope.....                         | 3 |
| 2. Overall Description .....                    | 3 |
| 2.1. Product Perspective.....                   | 3 |
| 2.2. User Classes and Characteristics .....     | 3 |
| 2.3. Design and Implementation Constraints..... | 3 |
| 3. System Features.....                         | 3 |
| 4. External Interface Requirements.....         | 4 |
| 4.1. User Interfaces .....                      | 4 |
| 4.2. Software Interfaces.....                   | 4 |
| 5. System Features/Modules.....                 | 4 |
| 5.1. Train Search and Filtering .....           | 4 |
| 5.2. Real-Time Seat Selection.....              | 4 |
| 5.3. Payment Processing .....                   | 4 |
| 5.4. Ticket Generation .....                    | 5 |
| 5.5. Booking Cancellation .....                 | 5 |
| 6. Nonfunctional Requirements.....              | 5 |
| 6.1. Performance .....                          | 5 |
| 6.2. Security .....                             | 5 |
| 6.3. Reliability .....                          | 5 |
| 7.UML UseCase diagram .....                     | 5 |
| 8.UML Class diagram .....                       | 6 |
| 9.UML Sequence diagram .....                    | 7 |



### Revision History

| Name          | Date       | Reason for Changes          | Version |
|---------------|------------|-----------------------------|---------|
| Jan Kośliński | 10.03.2026 | Initial version             | 1       |
| Jan Kośliński | 17.03.2026 | Adding UML UseCase diagram  | 2       |
| Jan Kośliński | 24.03.2026 | Adding UML Class diagram    | 3       |
| Jan Kośliński | 14.04.2026 | Adding UML Sequence diagram | 4       |

# 1. Introduction

## 1.1. Product Scope

The modern traveler demands efficiency, reliability, and transparency when planning journeys, yet the current train ticketing systems are frequently plagued by outdated interfaces, delayed data synchronization, and frustrating user experiences. Passengers often struggle to visualize train layouts, leading to dissatisfaction when assigned undesirable seats (e.g., backward-facing or away from travel companions). Furthermore, high traffic during peak holiday booking seasons frequently results in system crashes, slow loading times, and the dreaded "double-booking" scenario where two users attempt to reserve the same seat simultaneously because the system failed to lock the selection in real time.

RailReserve aims to solve these pain points by providing a highly responsive, mobile-first seat reservation application. The primary goal is to offer an interactive, real-time graphical seat map that allows users to seamlessly select, temporarily lock, and purchase their preferred seats with absolute confidence. By integrating secure payment gateways, instant digital ticket delivery (via QR code), and real-time inventory management, RailReserve will eliminate booking conflicts, reduce customer support overhead, and provide a frictionless travel planning experience from search to boarding.

## 2. Overall Description

### 2.1. Product Perspective

RailReserve is a standalone, mobile-first application designed to provide a real-time interface for train seat reservations. The system interacts with a central database to synchronize seat availability and integrates with external payment providers.

### 2.2. User Classes and Characteristics

**Passengers:** The primary users who search for trips, select seats on a graphical map, and manage their bookings.

### 2.3. Design and Implementation Constraints

**Security:** All transactions and personal data must be encrypted using TLS 1.3.

**Performance:** The system must handle real-time seat locking within strict time limits (10-minute window).

## 3. System Features

Based on the functional requirements, the system consists of the following core modules:

- Search Engine: Querying schedules and displaying available trains.

- Real-Time Seat Mapper: Visual selection and temporary locking of seats.
- Payment Processor: Handling secure financial transactions.
- Ticket Generator: Instant creation of digital tickets with QR codes.
- Booking Manager: Handling cancellations and seat status updates.

## 4. External Interface Requirements

### 4.1. User Interfaces

The system provides a highly responsive, mobile-first graphical interface. The core component is an interactive seat map that allows users to visualize the train layout and select seats by tapping.

### 4.2. Software Interfaces

Payment Gateway: Secure API connection for transmitting payment data and receiving transaction status.

Central Database: Real-time connection for updating seat statuses (e.g., from "Reserved" to "Available").

## 5. System Features/Modules

Here are five detailed functional requirements associated with the core features of the system:

### 5.1. Train Search and Filtering

- **Stimulus/Condition:** The user enters an origin station, destination station, and travel date, then presses "Search".
- **Response:** The system displays a list of available trains matching the criteria.
- **REQ-1.1:** Upon receiving the search criteria, the system shall query the train schedule database and return a chronological list of available trains within 2 seconds.

### 5.2. Real-Time Seat Selection

- **Stimulus/Condition:** The user taps on an available seat icon on the graphical train map.
- **Response:** The system highlights the seat, adds it to the user's cart, and locks it for others.
- **REQ-2.1:** Upon the user selecting an available seat, the system shall place a temporary hold on the seat for 10 minutes, changing its status to "Reserved" for all other users viewing the map.

### 5.3. Payment Processing

- **Stimulus/Condition:** The user enters payment details and submits the booking.
- **Response:** The system processes the transaction and confirms the booking.

- **REQ-3.1:** Upon submission of payment details, the system shall securely transmit the data to the payment gateway and process the transaction.
- **REQ-3.2:** The system shall handle payment failure by returning the user to the payment screen with an appropriate error message and maintaining the seat lock if the 10-minute window has not expired.

## 5.4. Ticket Generation

- **Stimulus/Condition:** The payment gateway returns a "Success" confirmation.
- **Response:** The system generates a digital ticket and displays it to the user.
- **REQ-4.1:** Upon successful payment confirmation, the system shall instantly generate a digital ticket containing a unique scannable QR code, travel details, and seat assignment.

## 5.5. Booking Cancellation

- **Stimulus/Condition:** The user selects an active ticket from their profile and clicks "Cancel Booking".
- **Response:** The system cancels the ticket and frees the seat in the database.
- **REQ-5.1:** Upon the user confirming a cancellation request, the system shall invalidate the QR code, initiate any applicable refund, and immediately update the seat's status back to "Available" in the central database

# 6. Nonfunctional Requirements

## 6.1. Performance

NF-1 : The interactive seat map shall load and reflect the current real-time availability of seats in under 1.5 seconds under normal network conditions.

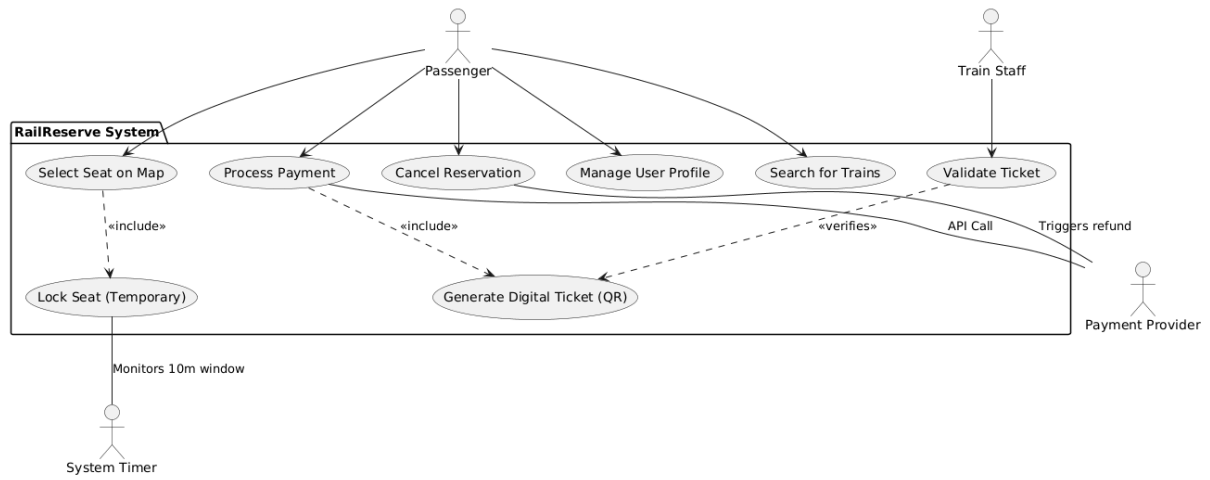
## 6.2. Security

NF-2: All user passwords and payment information shall be encrypted both in transit (using TLS 1.3) and at rest to ensure compliance with financial and data protection standards

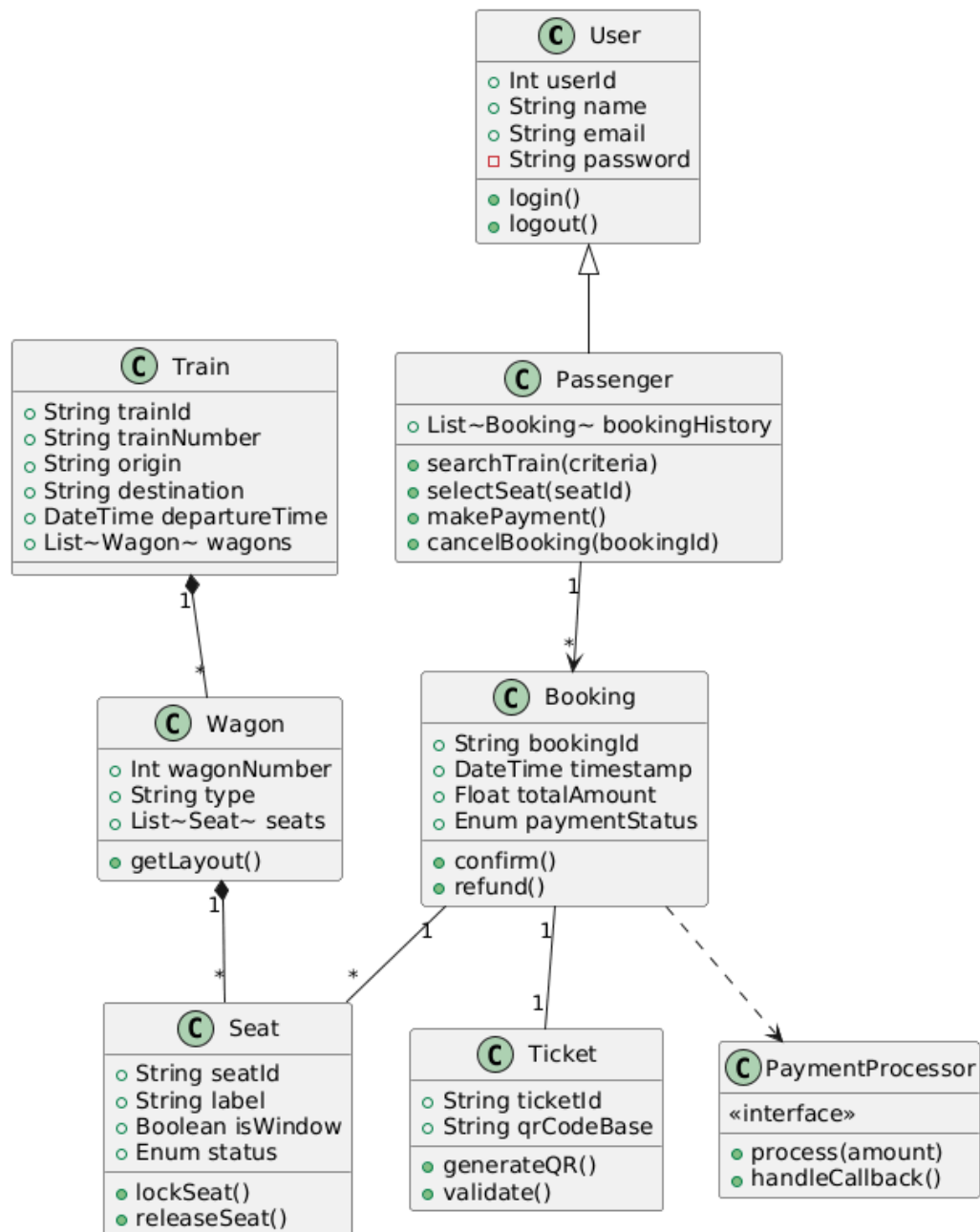
## 6.3. Reliability

NF-3: The core reservation system shall maintain a 99.9% uptime to ensure passengers can book tickets and access their digital QR codes at any time of day.

## 7.UML UseCase diagram



## 8.UML Class diagram



## 9.UML Sequence diagram

